

A statistical derivation of similarity templates

Thomas P. Minka

May 23, 2002

1 Introduction

Stauffer & Grimson (2001) is an example of how not to use statistics in computer vision. They took a simple heuristic algorithm and made it seem complicated by dressing up the explanation with probabilistic formulas that don't make sense. Furthermore, this "probabilizing" is applied separately to each step in their algorithm, leading to inconsistent assumptions. This paper shows how Stauffer&Grimson's algorithms can be obtained from a consistent set of probabilistic assumptions, plus some inspired approximations.

2 Similarity templates

Stauffer & Grimson (2001) is actually two papers: one about image matching (applied to detection) and another about image segmentation (applied to recognition). The common thread between them is the "similarity template" representation of images. The similarity template of an image $\mathbf{x}$ with pixels $\{x_i, i = 1, \dots, n\}$ is an $n \times n$ matrix $\mathbf{S}(\mathbf{x})$ with entries

$$S_{ij}(\mathbf{x}) = \frac{k(x_i - x_j)}{\sum_j k(x_i - x_j)} \quad (1)$$

$$k(x) = \exp(-||x||^2/v) \quad (2)$$

Each row of this matrix sums to 1, and represents an asymmetric affinity of pixel i with all other pixels. v is a tuning parameter. For a grayscale image, the pixels are scalars; for color or multispectral images, the pixels are vectors which are compared by Euclidean distance. The match score for two images is simply the inner product of their similarity templates:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{ij} S_{ij}(\mathbf{x}) S_{ij}(\mathbf{y}) = \frac{\sum_{ij} k(x_i - x_j) k(y_i - y_j)}{\left(\sum_j k(x_i - x_j)\right) \left(\sum_j k(y_i - y_j)\right)} \quad (3)$$

To perform detection, one averages the similarity templates of all positive examples and evaluates the match with patches of an image.

A similarity template, aka affinity matrix, can also be used to cluster the pixels and thereby segment the image. Define $p(c|i)$ to be the soft assignment of pixel i to cluster c and $p(c)$ to be the size of c , normalized over all clusters. By Bayes' rule, there is a corresponding quantity $p(i|c)$ which specifies how much of cluster c is devoted to pixel i . The pixels are clustered by

maximizing the following expression over cluster weights $p(c)$ and assignments $p(i|c)$:

$$E = \sum_{ij} S_{ij} \log \sum_c p(c) p(i|c) p(j|c) \quad (4)$$

At convergence, the assignments are thresholded to get a segmentation of the image. A single ‘average’ segmentation can be computed for a set of images by averaging together their similarity templates. The image set can then be indexed according to their average color in each region of the ‘average’ segmentation.

3 The image model

We will now show that a certain probabilistic image model leads to the above algorithms for matching and segmentation. The model says that an image is composed of regions whose colors can change across images but whose location and shape remain fixed.

Specifically, let r_{ci} be a binary variable which indicates whether pixel i is in region c . The colors in each region are assumed to be drawn independently from a color distribution $p(x|c, \theta)$, which lives in a family parameterized by θ . A key aspect of the model is that we regard θ as a missing variable which can vary from image to image according to some prior distribution $p(\theta)$. Putting these assumptions together gives the following formula for the probability of an image $\mathbf{x}$:

$$p(\mathbf{x}|\mathbf{r}) = \int_{\theta} p(\theta) \prod_i \sum_c r_{ci} p(x_i|c, \theta) d\theta \quad (5)$$

Note that the summation over c is just a way of picking out the c for which $r_{ci} = 1$. This formula makes explicit that the segmentation is fixed for all images but the appearance of each region can vary. Note that the model cannot capture the texture but only the color histogram of each region.

To understand how this model works, suppose the region distributions $p(x|c, \theta)$ were Gaussian, and θ specified separate means and variances for the regions. Since we know what pixels were generated by each region model, it is straightforward to integrate the parameters of each Gaussian, and the result is similar to plugging in an estimate of the parameters of that Gaussian. This is a general property of probabilistic inference: whether the model is Gaussian or non-Gaussian, integrating over θ is essentially the same as computing a density estimate of each region. Thus we can approximate (5) as follows:

$$p(\mathbf{x}|\mathbf{r}) \approx \prod_i \sum_c r_{ci} \hat{p}(x_i|c) \quad (6)$$

where $\hat{p}(x|c)$ is *any* density estimate for region c , based on the image $\mathbf{x}$. The prior $p(\theta)$ is implicit in the estimator used for $\hat{p}$. Note that while (5) can be used for sampling an image according to the model, (6) can only be used to compute the probability of a given image.

What density estimator should we use? Rather than fit a Gaussian, we will be more flexible and use a kernel estimate with kernel (2):

$$\hat{p}(x|c) = \frac{\sum_j r_{cj} k(x - x_j)}{\sum_j r_{cj}} \quad (7)$$

This completes the specification of the model, whose parameters are the segmentation $\mathbf{r}$ and kernel width v .

Unfortunately, this parameterization of the model is difficult to fit to data. Given a set of images, we would need to perform a combinatorial search over $\mathbf{r}$ to maximize likelihood. Thus we introduce a continuous extension of $\mathbf{r}$, by interpreting r_{ci} as $p(c|i)$, and $r_{cj} / \sum_k r_{ck}$ as $p(j|c)$:

$$p(\mathbf{x}|\mathbf{r}) \approx \prod_i \sum_c p(c|i) \sum_j p(j|c) k(x_i - x_j) \quad (8)$$

$$= \prod_i \sum_j \left(\sum_c p(j|c) p(c|i) \right) k(x_i - x_j) \quad (9)$$

$$= \prod_i \sum_j p(j|i) k(x_i - x_j) \quad (10)$$

In this parameterization, we allow $p(c|i)$ and $p(j|c)$ to be arbitrary probability vectors, related by Bayes' rule:

$$p(c|i) = \frac{p(i|c)p(c)}{p(i)} \quad (11)$$

This is the parameterization assumed in Stauffer&Grimson's algorithms. After fitting, the soft probabilities $p(c|i)$ are thresholded into hard assignments r_{ci} . Note that this is only interpreted as a reparameterization of the model. It should not be interpreted as a new model where pixels may switch regions with probability $p(c|i)$, since that leads to an integral over region assignments and therefore a different probability formula. It is analogous to the continuous extension used by the softassign algorithm for matching point sets.

4 Fitting the model

This section shows that the segmentation algorithm of Stauffer&Grimson is closely related to fitting the image model by maximum likelihood. First consider the case of a single image. The parameters of the model are $p(c)$ and $p(i|c)$ (which is the same as $p(j|c)$). For shorthand, we will sometimes write $p(i) = \sum_c p(c)p(i|c)$ and $p(j|i) = \sum_c p(c|i)p(j|c)$.

To maximize the likelihood, we will repeatedly maximize a lower bound, which is actually a

concatenation of two Jensen-type bounds. First apply the following bound:

$$\log L = \sum_i \log \left(\sum_j p(j|i) k(x_i - x_j) \right) \quad (12)$$

$$\geq \sum_{ij} S_{ij} \log p(j|i) k(x_i - x_j) + \text{const.} \quad (13)$$

$$S_{ij} = \frac{p(j|i)^{\text{old}} k(x_i - x_j)}{\sum_j p(j|i)^{\text{old}} k(x_i - x_j)} \quad (14)$$

At the beginning of the algorithm, $p(j|i)$ is $1/n$, so S_{ij} is the similarity template of the image. Ignoring terms that don't depend on the parameters, we now have to maximize

$$\sum_{ij} S_{ij} \log \sum_c p(c) p(i|c) p(j|c) - \sum_i \log p(i) \quad (15)$$

The term with $p(i)$ complicates things because it is a sum over products of the parameters. To get Stauffer&Grimson's algorithm, we approximate it as constant. This leaves only (4). To maximize it, apply another bound:

$$\sum_{ij} S_{ij} \log \sum_c p(c) p(i|c) p(j|c) \quad (16)$$

$$\geq \sum_{ij} S_{ij} \sum_c q_{ij}(c) \log p(c) p(i|c) p(j|c) + \text{const.} \quad (17)$$

$$q_{ij}(c) = \frac{p(c) p(i|c)^{\text{old}} p(j|c)^{\text{old}}}{\sum_c p(c) p(i|c)^{\text{old}} p(j|c)^{\text{old}}} \quad (18)$$

Solving for $p(c)$:

$$p(c) = \frac{1}{n} \sum_{ij} S_{ij} q_{ij}(c) \quad (19)$$

The remaining terms simplify as follows:

$$\sum_{ci} \left(\sum_j S_{ij} q_{ij}(c) \right) \log p(i|c) + \sum_{cj} \left(\sum_i S_{ij} q_{ij}(c) \right) \log p(j|c) \quad (20)$$

$$= \sum_{ci} \left(\sum_j S_{ij} q_{ij}(c) + S_{ji} q_{ji}(c) \right) \log p(i|c) \quad (21)$$

Note that $q_{ji}(c) = q_{ij}(c)$, but $S_{ji} \neq S_{ij}$.

$$\frac{d}{dp(i|c)} = \frac{\sum_j (S_{ij} + S_{ji}) q_{ij}(c)}{p(i|c)} + \lambda \quad (22)$$

$$p(i|c) \propto \sum_j (S_{ij} + S_{ji}) q_{ij}(c) \quad (23)$$

This algorithm has two E-steps, (14) and (18). Stauffer&Grimson iterate (18) but always leaves S_{ij} at its starting value. Stauffer&Grimson are thus optimizing a *quasi-likelihood* $\underline{L}$:

$$\log \underline{L} = \sum_{ij} S_{ij} \log \frac{p(j|i)k(x_i - x_j)}{S_{ij}} \quad (24)$$

$$S_{ij} = \frac{k(x_i - x_j)}{\sum_j k(x_i - x_j)} \quad (25)$$

Note that this algorithm can be applied to cluster any kind of data, not just pixels. If the region models were Gaussian, fitting the model would be equivalent to k-means. By using kernel estimates for the regions, this algorithm generalizes k-means to allow clusters of arbitrary shape, yet retains a probabilistic interpretation.

5 Image matching

For image matching, we apply the same model but instead of maximum likelihood we will perform Bayesian averaging over $\mathbf{r}$. Given two images $\mathbf{x}$ and $\mathbf{y}$, the match score will be the probability that they came from the same model, i.e. that they share the same $\mathbf{r}$. This probability can be written as follows:

$$p(\text{same } \mathbf{r} | \mathbf{x}, \mathbf{y}) = \frac{p(\mathbf{x}, \mathbf{y} | \text{same } \mathbf{r})p(\text{same } \mathbf{r})}{p(\mathbf{x}, \mathbf{y} | \text{same } \mathbf{r})p(\text{same } \mathbf{r}) + p(\mathbf{x}, \mathbf{y} | \text{different } \mathbf{r})p(\text{different } \mathbf{r})} \quad (26)$$

which is a monotonic function of the ratio

$$d(\mathbf{x}, \mathbf{y}) = \frac{p(\mathbf{x}, \mathbf{y} | \text{same } \mathbf{r})}{p(\mathbf{x}, \mathbf{y} | \text{different } \mathbf{r})} = \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \quad (27)$$

$$p(\mathbf{x}) = \int_{\mathbf{r}} p(\mathbf{x} | \mathbf{r})p(\mathbf{r})d\mathbf{r} \quad (28)$$

$$p(\mathbf{x}, \mathbf{y}) = \int_{\mathbf{r}} p(\mathbf{x} | \mathbf{r})p(\mathbf{y} | \mathbf{r})p(\mathbf{r})d\mathbf{r} \quad (29)$$

An elegant approximation gives us a formula for $d(\mathbf{x}, \mathbf{y})$ in terms of the similarity templates $S(\mathbf{x})$ and $S(\mathbf{y})$. First, switch to the continuous parameters $p(j|i)$ instead of $\mathbf{r}$. Now ignore the symmetry constraints and let $p(j|i)$ for different i be independent under the prior, with $E[p(j|i)] = 1/n$. This immediately tells us that

$$p(\mathbf{x}) = \prod_i \sum_j E[p(j|i)]k(x_i - x_j) \quad (30)$$

$$= \prod_i \sum_j \frac{1}{n} k(x_i - x_j) \quad (31)$$

which is an intuitively appealing formula. It gives high probability to images with a small number of compact clusters. Equivalently, it gives high probability to images whose color histogram has low entropy—more on this below.

Next we will get an approximation for $p(\mathbf{y}|\mathbf{x})$. Given $\mathbf{x}$, the posterior for $p(j|i)$ can be approximated by using the quasi-likelihood $\underline{L}$. The quasi-likelihood suggests that the posterior for $p(j|i)$ is Dirichlet, independently for each i , with mean

$$E[p(j|i)|\mathbf{x}] = S_{ij} \quad (32)$$

From this we can derive the conditional probability of image $\mathbf{y}$ given image $\mathbf{x}$:

$$p(\mathbf{y}|\mathbf{x}) = \int_{\mathbf{r}} p(y|\mathbf{r})p(\mathbf{r}|\mathbf{x})d\mathbf{r} \quad (33)$$

$$\approx \prod_i \sum_j E[p(j|i)|\mathbf{x}]k(y_i - y_j) \quad (34)$$

$$= \prod_i \frac{\sum_j (1/n)k(x_i - x_j)k(y_i - y_j)}{\sum_j (1/n)k(x_i - x_j)} = \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})} \quad (35)$$

This implies that

$$p(\mathbf{x}, \mathbf{y}) = \prod_i \sum_j (1/n)k(x_i - x_j)k(y_i - y_j) \quad (36)$$

which is also intuitive. It is the same kind of expression as $p(\mathbf{x})$ but where the observations are now pixel pairs (x_i, y_i) . For three images, we expect a similar formula using pixel triples, and so on.

From this we obtain

$$\frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} = \prod_i \frac{\sum_j (1/n)k(x_i - x_j)k(y_i - y_j)}{\left(\sum_j (1/n)k(x_i - x_j)\right) \left(\sum_j (1/n)k(y_i - y_j)\right)} \quad (37)$$

$$= \prod_i n \sum_j S_{ij}(\mathbf{x})S_{ij}(\mathbf{y}) \quad (38)$$

which is similar to Stauffer&Grimson's formula (3). However, Stauffer&Grimson's formula is likely to be more robust than (37), and can be justified by that reason.

The Bayesian matching score (37) is equivalent to the mutual information score proposed by Voila, where all of the entropy expressions are approximated by kernel estimates:

$$\mathcal{H}(\mathbf{x}) \approx -\frac{1}{n} \sum_i \log \sum_j (1/n)k(x_i - x_j) \quad (39)$$

$$\mathcal{H}(\mathbf{x}, \mathbf{y}) \approx -\frac{1}{n} \sum_i \log \sum_j (1/n)k(x_i - x_j)k(y_i - y_j) \quad (40)$$

$$\mathcal{I}(\mathbf{x}, \mathbf{y}) = \mathcal{H}(\mathbf{x}) + \mathcal{H}(\mathbf{y}) - \mathcal{H}(\mathbf{x}, \mathbf{y}) \quad (41)$$

This connection is interesting because it shows that the mutual information score can be interpreted as a Bayesian score for a particular image model, and can be implemented via template matching.

References

Stauffer, C., & Grimson, E. (2001). Similarity templates for detection and recognition. *CVPR'2001*.